

Árvore B em Memória Secundária

Implementação de uma classe Árvore B de ordem m operando estritamente em disco

Algoritmos e Estruturas de Dados — AED-PG-2026 (5955001-3)

Prof. Dr. José Augusto Baranauskas — DCM / USP

Linguagem: C++17 · 1º Semestre/2026

Alunos: Cássio Takarada · Cézio Luiz Ferreira Junior

2. Roteiro da apresentação

Parte 1 — a estrutura

- Contexto: por que disco e por que Árvore B.
- Como um nó é organizado e como o arquivo é gravado.
- Decisões de implementação (1 nó por I/O).
- Os métodos: busca, inserção e remoção.
- Demonstração ao vivo + grafos gerados.

Parte 2 — a avaliação

- Experimentos, métricas e metodologia.
- Impacto da ordem m (I/O e altura).
- Resultados por operação e por tamanho do conjunto.
- Reaproveitamento de nós (espaço).
- Validação em dois sistemas + discussão (union) e conclusões.

3. O problema e os objetivos

- Enunciado: implementar uma classe Árvore B de ordem m que resida em MEMÓRIA SECUNDÁRIA (um arquivo em disco).
- Restrição central: a árvore é lida/transferida em blocos — carregada parcialmente, NUNCA inteira na RAM. Cada operação toca um nó por vez.
- Operações exigidas: busca, inserção e remoção completas, mantendo a árvore balanceada.
- Instrumentação pedida: contar acessos ao disco e permitir monitorar a estrutura (impressão, altura, grafo).
- Nosso objetivo extra: avaliar empiricamente o efeito de m e do tamanho do conjunto, e validar os resultados em mais de uma máquina.

4. Contexto — memória rápida × disco lento

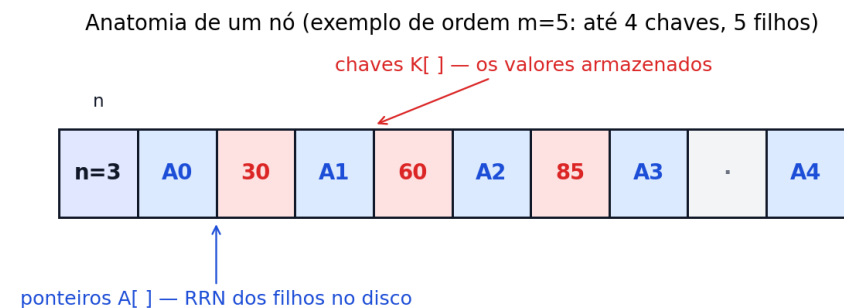
- A memória RAM é rapidíssima, mas é pequena e volátil (some quando desliga).
- O disco é enorme e permanente, porém MILHARES de vezes mais lento que a RAM.
- Num banco de dados, o gargalo quase sempre é o disco — não a CPU.
- Por isso a regra do trabalho: a árvore mora no disco e nunca é carregada inteira; cada passo lê ou escreve só um pedaço (um nó).
- Consequência: para ser rápido, o segredo é fazer o MENOR número de acessos ao disco possível.

5. Contexto — índice e a métrica honesta

- Um índice é o "sumário" que evita ler o arquivo inteiro: ele diz onde está cada registro.
- A Árvore B é a estrutura de índice usada de fato por bancos de dados (MySQL, PostgreSQL) e por sistemas de arquivos.
- A métrica que realmente importa NÃO é o relógio, e sim o NÚMERO DE ACESSOS AO DISCO — independe do hardware.
- Por que o relógio engana: a escrita vai para o cache do sistema operacional, então o tempo medido varia com a máquina e a carga.
- Toda a apresentação gira em torno de uma pergunta: como organizar o arquivo para responder buscas com o menor número de acessos?

6. Anatomia de um nó

- Cada nó é um registro de TAMANHO FIXO, gravado e lido de uma vez só.
- Campos: n (quantas chaves o nó tem), o vetor de ponteiros $A[]$ (os RRN dos filhos) e o vetor de chaves $K[]$ (os valores).
- Um nó de ordem m guarda até $m-1$ chaves e até m filhos — por isso m maior = mais chaves por nó.
- $PAGE_SIZE = 1 \text{ inteiro} + m \text{ ponteiros} + m \text{ chaves}$. Esse tamanho fixo é o que permite achar qualquer nó por cálculo direto.
- Regra de ouro: ler ou escrever UM nó = UM acesso ao disco.

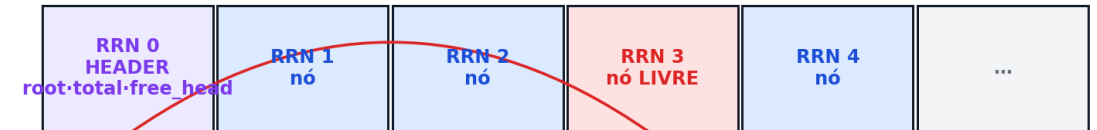


Registro de tamanho fixo: 1 inteiro (n) + m ponteiros + m chaves = $PAGE_SIZE$ bytes $\rightarrow$ 1 nó = 1 acesso ao disco

7. Layout do arquivo no disco

- O arquivo é um VETOR de registros de tamanho fixo, numerados por RRN (Relative Record Number).
- O registro 0 é o HEADER: guarda a raiz (root), o total de nós e a cabeça da lista de livres (free_head).
- Para achar um nó: $\text{offset} = \text{RRN} \times \text{PAGE_SIZE}$. Acesso direto, sem varrer o arquivo.
- Nós removidos não somem: entram numa lista de livres encadeada no próprio arquivo, prontos para reuso.
- Abrir o arquivo e já saber a raiz (no header) custa uma única leitura de metadado.

Layout do arquivo: um vetor de registros de tamanho fixo

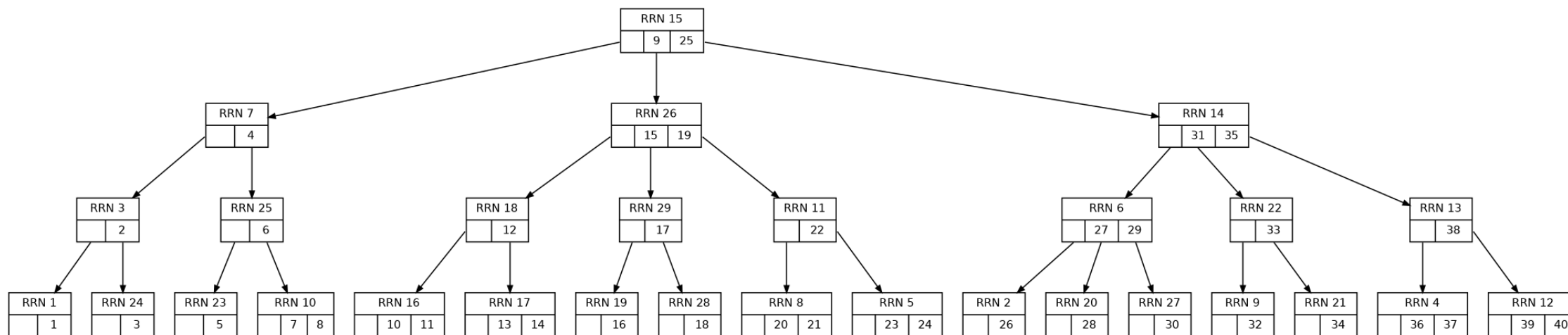


posição no arquivo: $\text{offset}(\text{rrn}) = \text{rrn} \times \text{PAGE_SIZE}$ (acesso direto, sem varrer nada)

free_head → lista de nós livres (reaproveitamento)

8. Decisões de implementação

- Ordem m é constante simbólica de compilação (M): estrutura totalmente parametrizada.
- Raiz da árvore: armazenada no header (registro 0) do arquivo — campo root.
- Um nó por I/O: readNode/writeNode de um registro de PAGE_SIZE; a árvore NUNCA é carregada inteira em memória.
- Registro do nó: $n \cdot A[0..m-1]$ (ponteiros RRN) $\cdot K[0..m-1]$ (chaves).
- Reaproveitamento de nós: estrutura = lista encadeada de livres (pilha LIFO) no próprio arquivo - free_head no header; cada nó livre marca $n=-1$ e aponta o próximo em $K[1]$; allocNode reusa antes de estender o arquivo.



Árvore B ($m=3$) com 40 chaves — exportada do nosso código (Graphviz); cada caixa mostra o RRN do nó em disco

9. O DiskManager — um nó por I/O

- Toda a conversa com o disco passa por uma única classe: o DiskManager.
- `readNode(rrn)` / `writeNode(rrn)`: leem e escrevem exatamente UM registro de `PAGE_SIZE` — nunca mais que isso.
- É aqui que mora o CONTADOR DE ACESSOS: cada `readNode/writeNode` soma 1. Essa é a métrica central da avaliação.
- `readHeader` / `writeHeader`: acessam o registro 0 (raiz, total, `free_head`).
- Garantia de projeto: a árvore nunca é carregada inteira — a lógica da B-tree só enxerga o disco por meio do DiskManager.

10. Reaproveitamento — a lista de livres

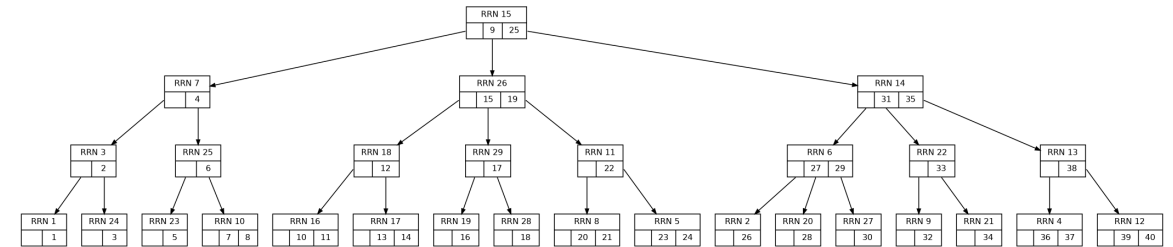
- Quando uma remoção libera um nó, ele não é descartado: vira um nó LIVRE.
- `freeNode(rrn)`: marca o nó com $n = -1$ e o encadeia na lista de livres, cuja cabeça (`free_head`) fica no header.
- `allocNode()`: antes de crescer o arquivo, pega o primeiro nó da lista de livres (pilha LIFO) — só estende o arquivo se não houver livres.
- Resultado: o arquivo reaproveita espaço em vez de inchar a cada remoção+inserção.
- É um comportamento ligável (`setReuse`) — usamos isso para medir o ganho no experimento de churn.

11. Métodos — visão geral

- Três operações principais, todas operando em disco: busca, inserção e remoção.
- `mSearch` / `mSearchPath` — busca m-way da raiz à folha; retorna (RRN, índice, achou).
- `insertB` — inserção bottom-up com propagação de split.
- `deleteB` — remoção com sucessor in-order, redistribuição (empréstimo) e fusão (merge).
- Auxiliares: `splitNode`, `insertInNode`, `removeFromNode`, `findSuccessorLeaf`.
- Monitoramento: `printTree` (hierárquico), `height`, `exportDot` (Graphviz) e o contador de acessos.
- Nos próximos slides, cada operação em detalhe.

12. Busca m-way

- Em cada nó, as chaves dividem o universo em FAIXAS — como um dicionário que diz 'entre tal e tal palavra'.
- Comparamos a chave buscada com as chaves do nó e descemos pelo ponteiro da faixa certa.
- Repetimos nó a nó até achar a chave ou chegar a uma folha sem ela.
- Custo: aproximadamente a ALTURA da árvore em acessos ao disco — pouquíssimos, porque a árvore é rasa.
- No grafo: buscar 70 desce da raiz (40, 60) para o filho da direita — 2 acessos.



Árvore B de ordem 3 — a busca segue um caminho da raiz à folha

13. Inserção e split

- A inserção é bottom-up: primeiro a busca encontra a FOLHA correta, depois inserimos a chave em ordem.
- Se o nó couber, acabou. Se ESTOURAR (passa de $m-1$ chaves), fazemos o split.
- Split: o nó cheio é dividido em dois e a chave do MEIO (mediana) SOBE para o nó pai.
- Se o pai também estourar, o split se propaga para cima — e pode até criar uma nova raiz, aumentando a altura.
- É exatamente esse mecanismo que mantém a árvore sempre BALANCEADA, sem precisar reorganizar tudo.

14. Remoção — sucessor, redistribuição e fusão

- É a operação mais delicada. Se a chave está num nó INTERNO, trocamos pelo seu sucessor in-order (a menor chave à direita), reduzindo o caso ao de uma folha.
- Ao remover de uma folha, o nó pode ficar ABAIXO DO MÍNIMO de chaves (underflow).
- Reparo 1 — redistribuição: se um irmão tem chaves sobrando, pegamos uma emprestada (passando pelo pai).
- Reparo 2 — fusão: se nenhum irmão pode emprestar, fundimos o nó com um irmão e puxamos uma chave do pai.
- A fusão pode propagar o underflow para cima — no limite, a raiz encolhe e a árvore fica mais baixa.

15. Resumo — tudo que o código faz

Núcleo — Árvore B em disco

- Árvore B de ordem m parametrizável (M em tempo de compilação).
- Opera 100% em disco: 1 nó por I/O; nada é carregado em RAM.
- Busca m -way (`mSearch` / `mSearchPath`).
- Inserção bottom-up com split (`insertB`).
- Remoção com sucessor, redistribuição e fusão (`deleteB`).
- Persistência: header (`root`, `total`, `free_head`) + nós de tamanho fixo, acesso direto por RRN.

Extras / funcionalidades acessórias

- Contador de acessos ao disco (métrica central da avaliação).
- Reaproveitamento de nós via free list, com liga/desliga (`setReuse`).
- Impressão hierárquica (`printTree`) e cálculo de altura (`height`).
- Exportação Graphviz (`exportDot`) -> imagens PNG dos grafos.
- Medição de ocupação: `total_nodes`, `free_nodes`, `fileSizeBytes`.
- Benchmark (`bench`): CSV com I/O, tempo CPU/IO (`getrusage`), altura, nós, tamanho.
- Automação: `run_all.sh` + `make_tables.py` (tabelas/gráficos) + `compare.py` (2 máquinas).
- Testes de stress + menu interativo (CRUD + métricas).

16. Demonstração — o programa rodando

- Animação real: do `make M=3` (compilação) até cada item do menu da main.
- Inserir · Buscar · Remover · ver Acessos ao disco · Imprimir árvore · Exportar grafo.
- Repetimos com $m=5$: nós comportam mais chaves, a árvore fica mais rasa.
- Tudo capturado da execução de verdade — sem montagem.

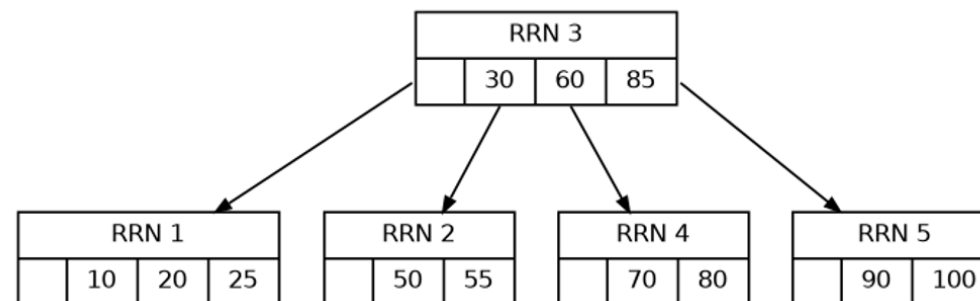
```
Arvore B (ordem m=5) - demo

$ ./btree demo5.bin

=== B-Tree (ordem 5) ===
... inserir / buscar / remover ...

> Imprimir arvore:
[RRN 1] (n=3) keys: 10 20 25
[RRN 2] (n=2) keys: 50 55
[RRN 4] (n=2) keys: 70 80
[RRN 5] (n=2) keys: 90 100
```

Grafo gerado (m=5)



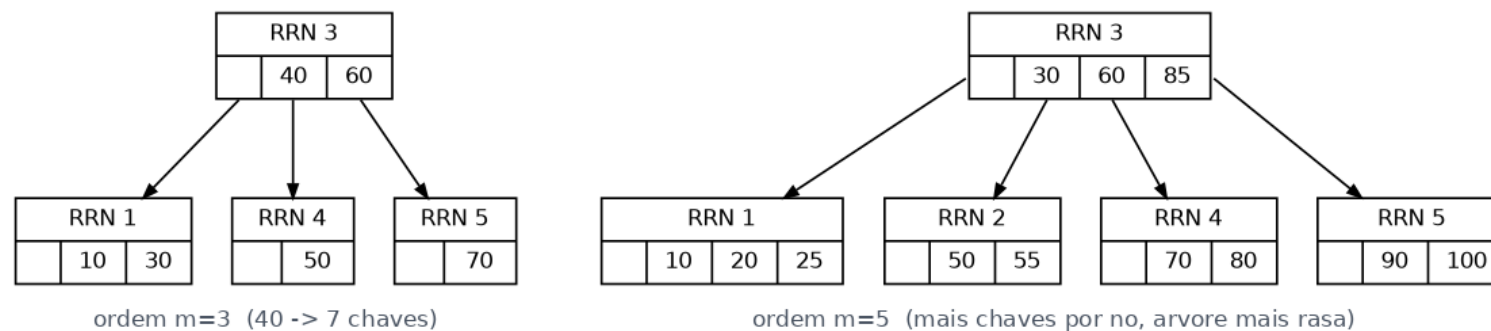
cada caixa = um nó em disco (RRN)
exportado pelo próprio programa -> exportDot() + Graphviz (dot)

make M=3/M=5 → menu interativo → grafo exportado pelo próprio programa

17. Grafos gerados pelo programa

- O programa exporta a árvore para Graphviz (exportDot) e gera a imagem do grafo.
- Cada caixa é um nó GRAVADO NO DISCO — o número é o RRN (a posição do registro no arquivo).
- $m=3$: no máximo 2 chaves por nó $\rightarrow$ mais níveis. $m=5$: até 4 chaves por nó $\rightarrow$ árvore mais rasa, menos acessos.

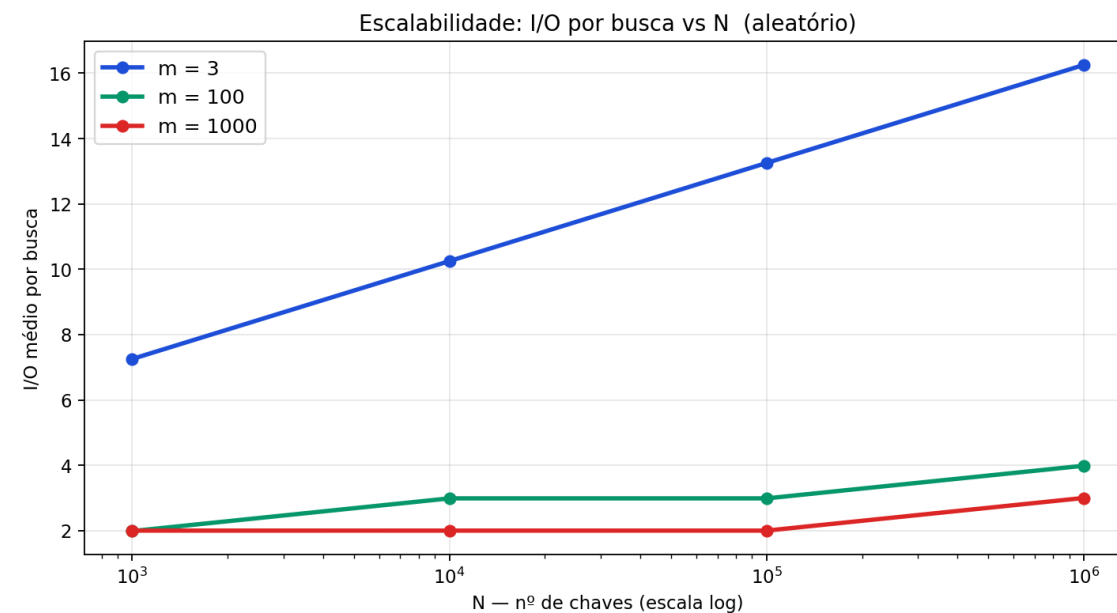
Grafos gerados pelo programa (exportDot + Graphviz)



Mesma sequência de chaves, duas ordens m — à direita a árvore é mais rasa

18. Os experimentos — a matriz de testes

- Exp. 1 - Impacto da ordem m: m em {3,4,5,8,16,32,64,100,128,256,512,1000}, $N=10^5$.
- Exp. 2 - Escala do conjunto: N em $\{10^3, 10^4, 10^5, 10^6\}$, m em {3,100,1000}.
- Exp. 3 - Ocupação do arquivo: churn COM e SEM reaproveitamento.
- Exp. 4 - Tempo: CPU (user+sys) vs espera de I/O (getrusage).
- Modos aleatório e sequencial em todos.
- Novidade: validação em 2 máquinas (Notebook x Titan/USP) - disco idêntico.

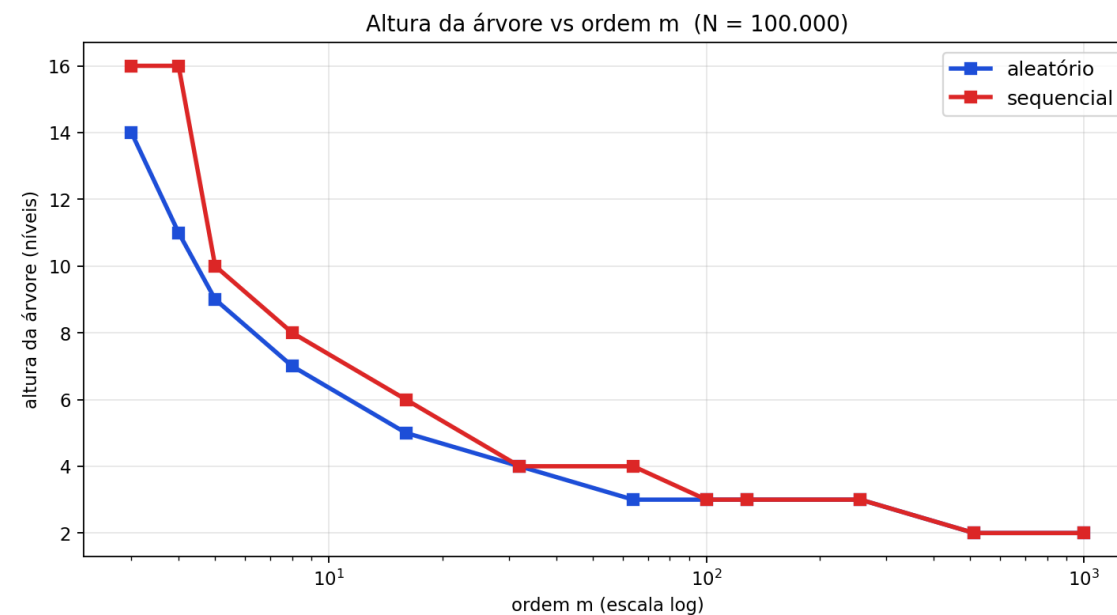


19. Metodologia e ambiente

- Driver não interativo (bench): roda inserção → busca → remoção e emite um CSV com todas as métricas.
- Cada configuração é reconstruída do zero (arquivo novo), garantindo medições independentes.
- Dois modos de carga: ALEATÓRIO (chaves embaralhadas) e SEQUENCIAL (em ordem) — estressam a árvore de formas diferentes.
- Tempo medido com getrusage: separa CPU de usuário, CPU de sistema (syscalls de I/O) e espera de I/O.
- Duas máquinas: meu Notebook e o servidor Titan (USP). Automação por run_all.sh + scripts Python.
- Métrica de referência sempre o contador de acessos — determinístico e comparável entre máquinas.

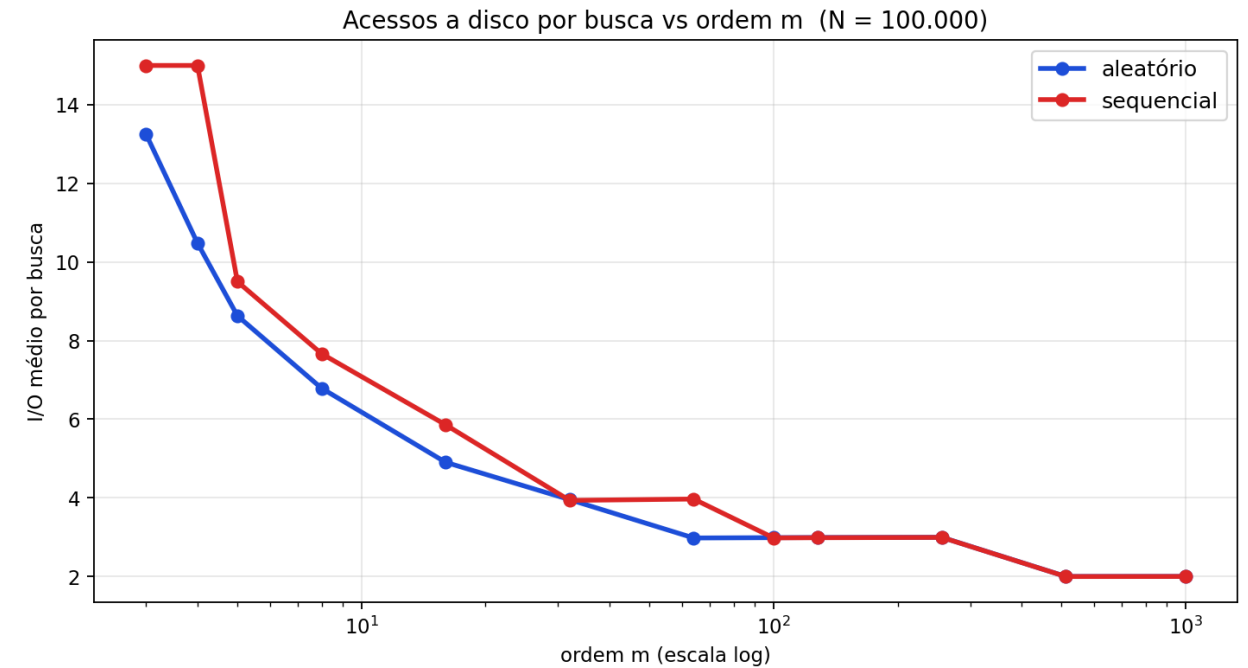
20. Métricas utilizadas

- Acessos ao disco - contador readNode+writeNode; total e média/op (métrica honesta de I/O).
- Altura da árvore - nº de níveis ($\sim \log_m N$).
- Ocupação - nós físicos, nós livres e tamanho em bytes.
- Tempo de parede (wall) - relógio de alta resolução.
- Tempo de CPU - usuário (lógica) e sistema (syscalls de I/O), via getrusage.
- Espera de I/O - wall menos CPU.



21. Impacto da ordem m — I/O por busca (N=10^5)

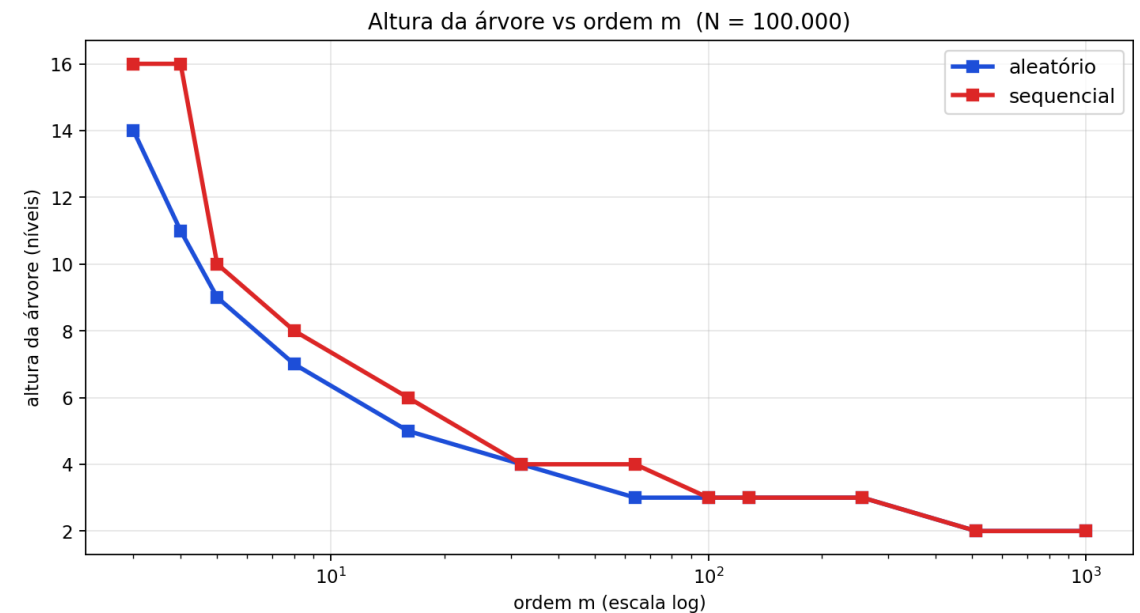
m	altura	busca I/O/op	inser. I/O/op	rem. I/O/op	arquivo (KB)
3	14	13.25	16.90	19.67	2,040
4	11	10.48	13.51	16.76	1,829
5	9	8.63	11.50	13.52	1,589
8	7	6.79	9.05	10.91	1,411
16	5	4.90	7.00	8.16	1,251
32	4	3.95	5.95	6.77	1,189
64	3	2.98	5.04	5.64	1,146
100	3	2.99	4.97	5.51	1,131
128	3	2.99	4.92	5.41	1,121
256	3	2.99	4.57	5.00	1,088
512	2	2.00	4.00	4.41	1,053
1000	2	2.00	3.99	4.34	1,016



I/O por busca cai de 13,25 ($m=3$, altura 14) para 2,00 ($m \geq 512$, altura 2). Ganho satura por volta de $m \sim 64-128$.

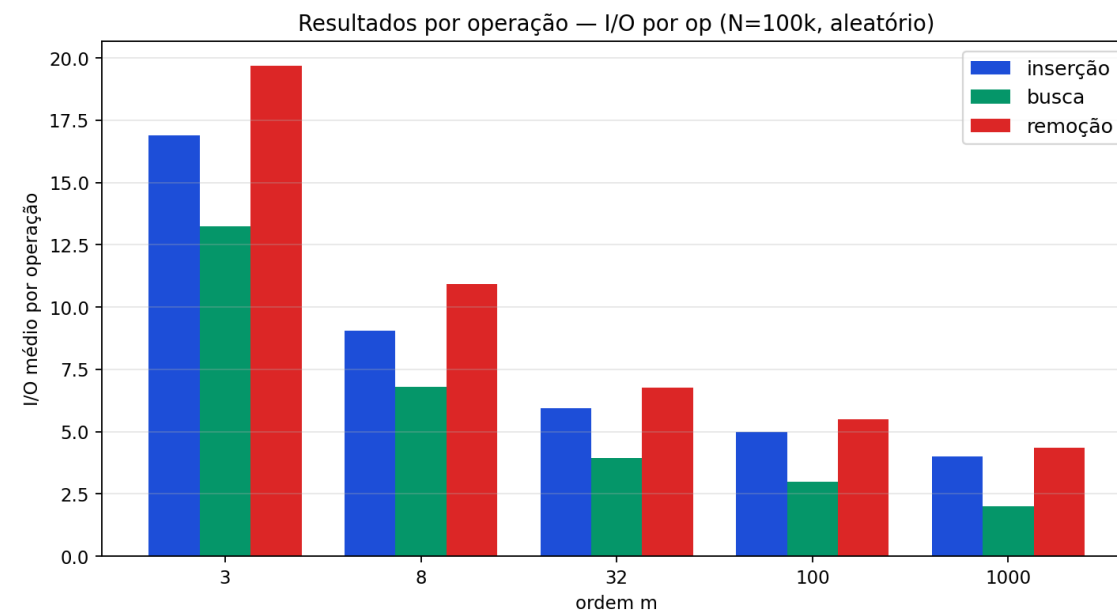
22. Impacto da ordem m — altura da árvore

- A altura é o número de níveis — e quase iguala o número de acessos por busca.
- Com $m=3$, a árvore tem 14 níveis para 100 mil chaves. É muito 'fundo' para uma estrutura de disco.
- Aumentando m , a altura DESPENCA em degraus: poucos níveis bastam, porque cada nó comporta muito mais chaves.
- A partir de m grande, a altura chega a 2 — não dá para ficar mais raso.
- Altura baixa = poucos acessos = a razão de existir da Árvore B.



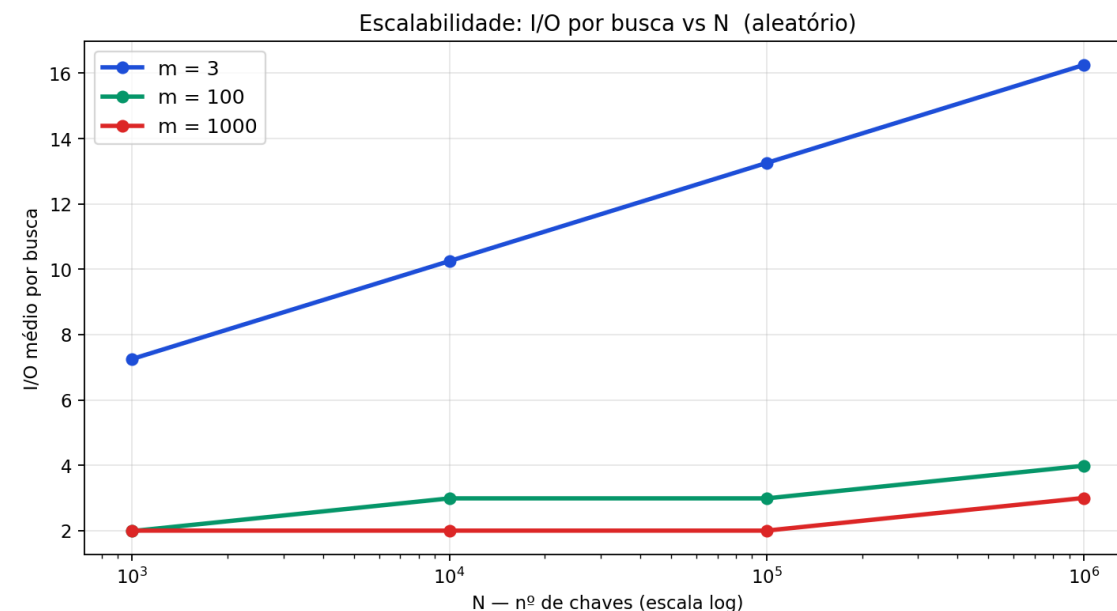
23. Resultados separados por operação

- Mesmo experimento, agora separando inserção, busca e remoção.
- A BUSCA é a mais barata: desce direto da raiz à folha (~altura acessos).
- INSERÇÃO custa um pouco mais: além de descer, pode reescrever nós no caminho (split).
- REMOÇÃO é a mais cara: pode ler irmãos para redistribuir/fundir nós (reparo de underflow).
- As três caem junto com m — e todas saturam quando a árvore chega a 2-3 níveis.
- Ou seja: o ganho de aumentar m vale para TODAS as operações, não só para a busca.



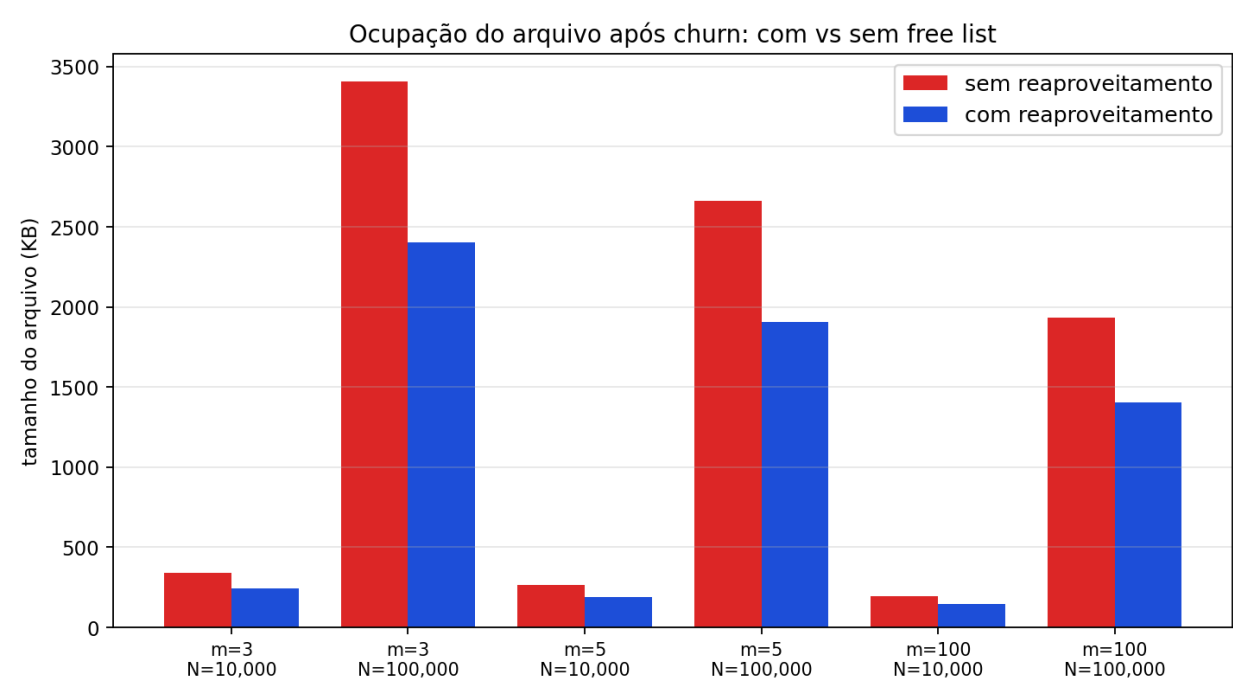
24. Escala do conjunto (N de mil a 1 milhão)

- Agora fixamos a ordem e variamos o tamanho do conjunto N, de mil até um MILHÃO de chaves.
- Mesmo multiplicando N por mil, o número de acessos por busca quase não muda.
- Motivo: o custo é logarítmico na base m — dobrar/multiplicar os dados acrescenta no máximo um nível.
- Com m grande, a curva fica praticamente plana: a estrutura aguenta bases enormes.
- É a tradução prática de 'escalável': cresce o dado, não cresce o custo por operação.



25. Reaproveitamento de nós — resultado (churn)

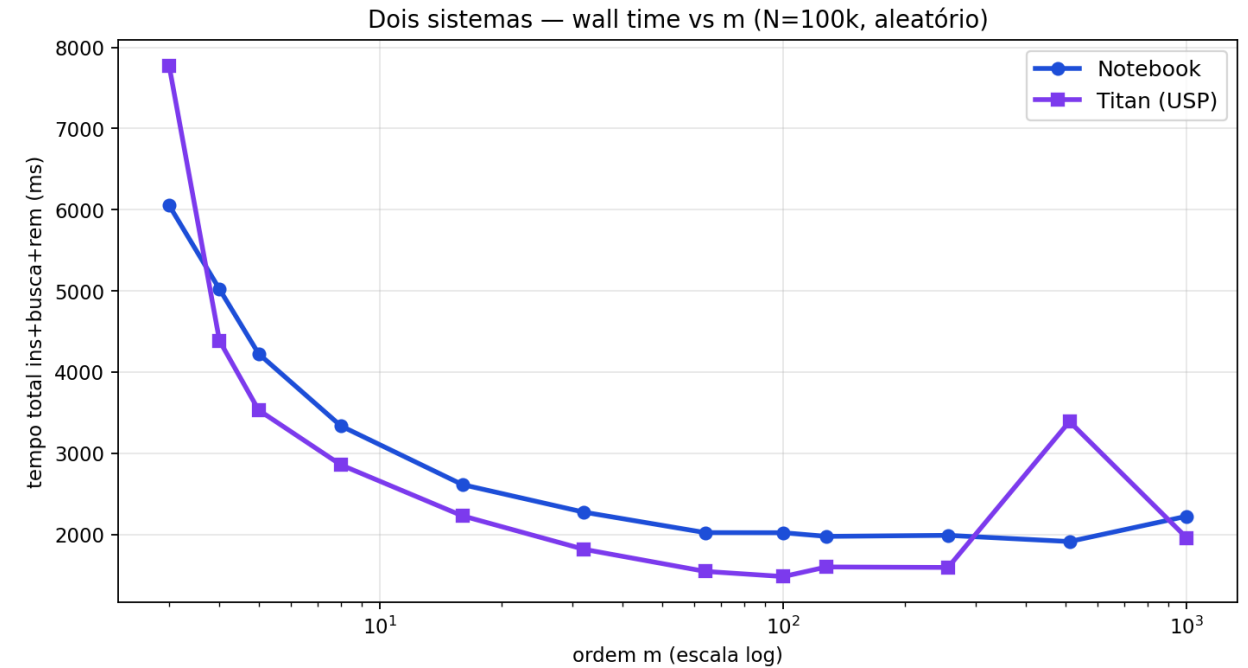
m	N	sem reuso (KB)	com reuso (KB)	economia
3	10,000	342	241	30%
3	100,000	3,407	2,404	29%
5	10,000	266	190	29%
5	100,000	2,663	1,905	28%
100	10,000	196	144	27%
100	100,000	1,931	1,404	27%



O reaproveitamento de nós economiza ~27-30% do tamanho do arquivo neste workload.

26. Dois sistemas — comparação de tempo (wall)

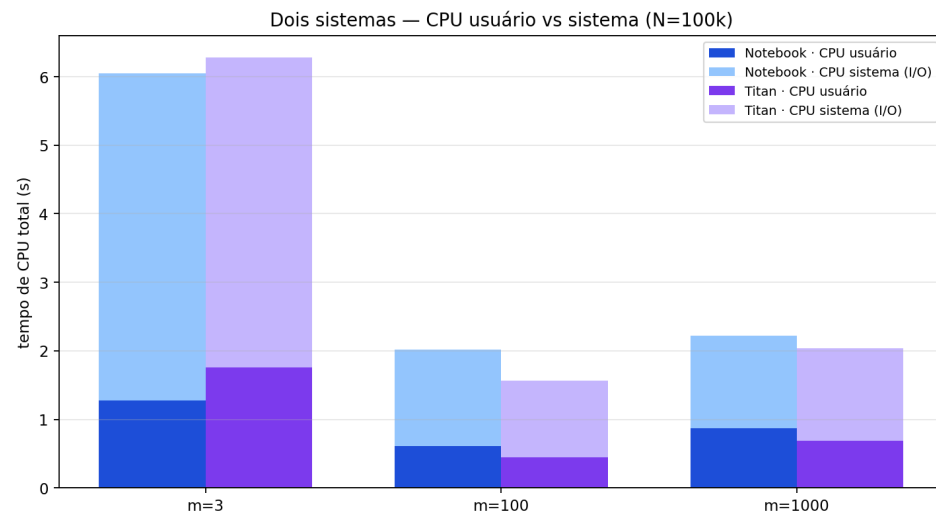
m	acessos disco	Notebook wall	Titan wall	speedup
3	4.982.252	6.05 s	7.78 s	0.78×
16	2.006.878	2.61 s	2.23 s	1.17×
100	1.346.869	2.02 s	1.48 s	1.36×
1000	1.033.255	2.22 s	1.96 s	1.14×



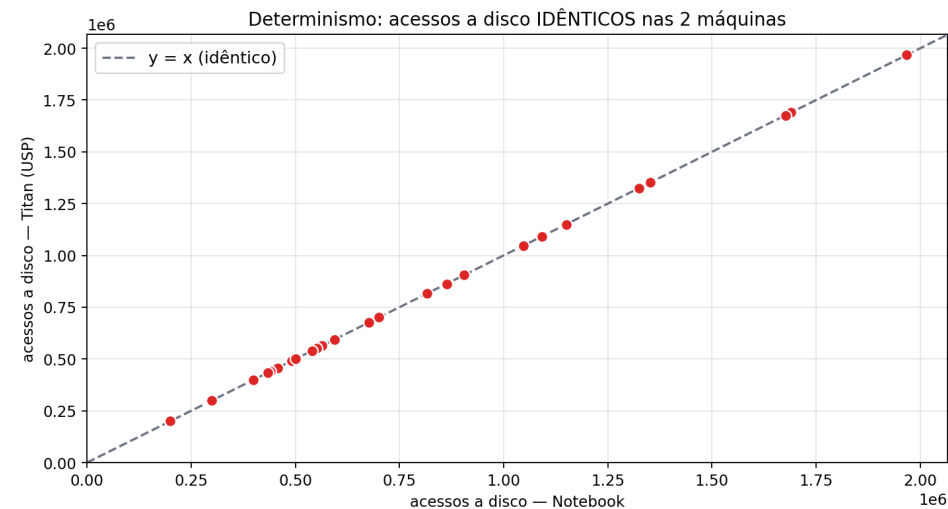
Mesmo programa, mesmo arquivo, duas máquinas. Acessos a disco idênticos; só o tempo muda — o Titan (USP) é mais rápido, mas a CURVA tem a mesma forma.

27. Dois sistemas — CPU e prova de determinismo

- Esquerda: o tempo é quase todo CPU de SISTEMA (chamadas de I/O ao SO), não espera de disco — efeito do cache de páginas. Por isso o relógio não é a métrica honesta.
- Direita: cada ponto é (acessos no Notebook, acessos no Titan). TODOS caem na reta $y=x$ — ou seja, o número de acessos é exatamente igual nas duas máquinas.
- Conclusão: a implementação é determinística e portátil; o hardware muda o tempo, nunca o comportamento lógico.



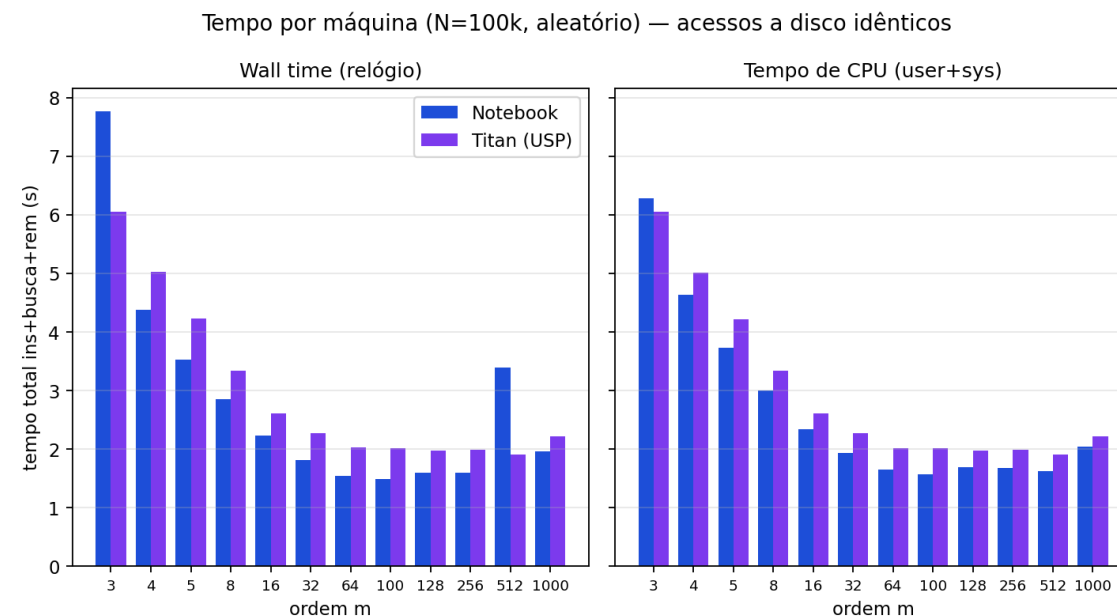
CPU usuário vs sistema (I/O) nas 2 máquinas



Acessos a disco idênticos — pontos sobre a reta $y=x$

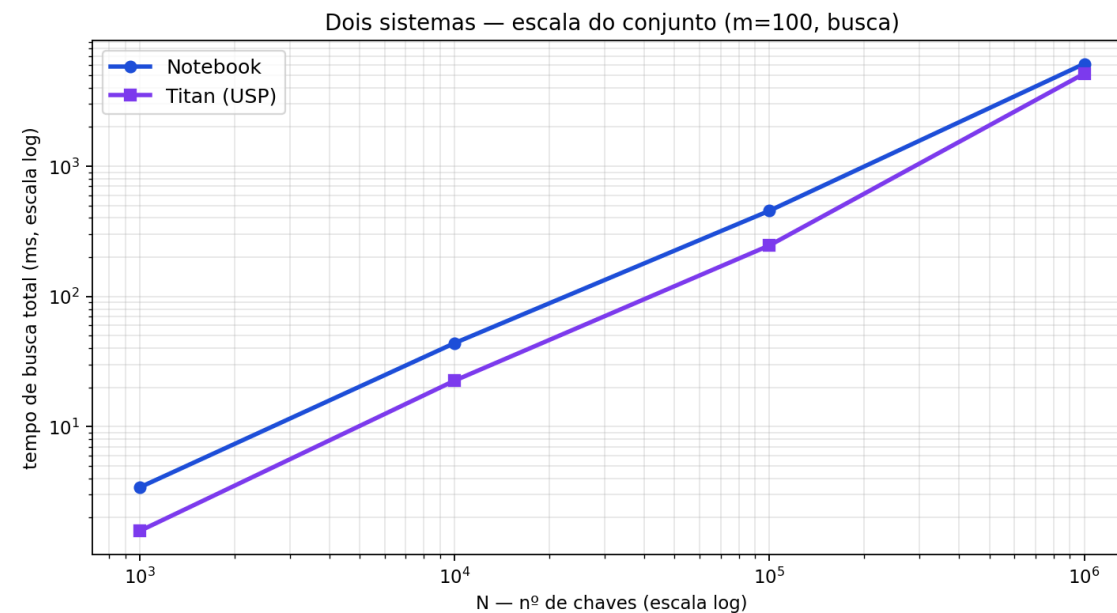
28. Dois sistemas — validação cruzada (resumo)

- Execução em duas máquinas: Notebook x Titan (USP).
- Acessos a disco IDÊNTICOS nas duas — resultado determinístico, independente do hardware.
- Apenas o tempo varia: wall time (inclui espera de I/O e carga) e tempo de CPU (user+sys, só o trabalho do processo).
- Comparação automatizada via `compare.py` a partir dos CSVs de cada máquina.



29. Dois sistemas — escala do conjunto

- Agora variando N de mil a um MILHÃO de chaves (ordem $m=100$), nas duas máquinas.
- Multiplicamos N por 1000 e o tempo de busca cresce devagar — porque a árvore é logarítmica na base m .
- As duas máquinas mantêm a mesma tendência; o Titan fica abaixo (mais rápido), em paralelo.
- É a prova prática de escalabilidade: a estrutura aguenta bases grandes sem explodir o custo.



30. E se usássemos union no header? (sugestão do prof.)

Como está hoje

- Header { root, total, free_head } e BNode { n, A[], K[] } são dois tipos SEPARADOS.
- Toda E/S serializa campo a campo com memcpy (readHeader/writeHeader/readNode/writeNode).
- O registro 0 (header) usa só 12 bytes, mas reserva uma página inteira (PAGE_SIZE) para manter o RRN uniforme: $\text{offset} = \text{rrn} \times \text{PAGE_SIZE}$.
- O nó LIVRE "esconde" o ponteiro do próximo em K[1] (com n = -1) — funciona, mas é implícito.

Com union { Header; BNode; FreeNode; }

- Um único tipo Página: todos os registros (header e nós) compartilham a MESMA página de tamanho fixo — garantido em tempo de compilação por sizeof.
- Lê/escreve a página inteira de uma vez e a interpreta como header (RRN 0) ou nó (RRN ≥ 1): some a serialização campo a campo → menos código e menos bugs de offset (justo as dificuldades que tivemos).
- A lista de livres ganha um campo explícito FreeNode.next no lugar do "K[1] mágico" → autodocumentado.
- Custo: gravar struct cru depende de layout/padding/endianness → menos portátil (a validação Notebook×Titan supõe layout igual); ler membro ≠ do escrito é UB em C++ — mitiga-se com memcpy / std::bit_cast.
- Veredito: página uniforme e código enxuto, em troca de cuidado com a portabilidade do binário.

31. Dificuldades técnicas

- Indexação 1-based dos nós (RRN 0 é o header): exigiu cuidado em todo cálculo de posição.
- Remoção: manter o caminho (path) na ordem certa para reparar o underflow foi a parte mais difícil.
- eofbit do fstream: uma leitura no fim do arquivo deixava o stream em estado de erro e fazia leituras seguintes falharem em silêncio — resolvido com clear().
- Garantir de verdade 1 nó por I/O: nenhuma estrutura podia 'esconder' a árvore em RAM.
- Serialização campo a campo (memcpy): funciona, mas é verbosa e propensa a erros de offset — daí a sugestão do union.

32. Vantagens × Desvantagens (trade-offs)

Vantagens

- Altura baixa → pouquíssimos acessos ao disco.
- Sempre balanceada, sem reorganização global.
- Ideal para memória secundária e índices de SGBD.
- Determinística: mesmo resultado em qualquer máquina.
- Reaproveitamento de espaço via lista de livres.

Desvantagens / custos

- Nós podem ficar subocupados (~50% no caso sequencial).
- Remoção é complexa (sucessor, redistribuição, fusão).
- Para m muito grande, varrer as chaves dentro do nó passa a custar CPU.
- Trade-off central: m maior → menos I/O, mas mais CPU por nó.
- Existe, portanto, um m ÓTIMO (nó do tamanho de um bloco de disco).

33. Aplicações práticas

- Índices de SGBDs: B-tree / B+ tree são a base dos índices de MySQL/InnoDB, PostgreSQL, Oracle e SQL Server.
- Sistemas de arquivos: NTFS, HFS+/APFS, ext4 (HTree) e Btrfs usam B-trees para diretórios e metadados.
- Armazenamento chave-valor / embarcado: BerkeleyDB, SQLite, LMDB.
- Onde eu usaria: como índice (primário ou secundário) de um banco de dados em disco - exatamente o cenário deste trabalho.

34. Ferramentas utilizadas

Implementação & experimentos

- Linguagem: C++17 (g++ -O2) — núcleo da Árvore B em disco.
- Build: GNU Make (parametriza a ordem via -DM=<m>).
- Persistência: std::fstream (arquivo binário, registros de tamanho fixo).
- Graphviz (dot): renderiza os grafos exportados pelo programa (exportDot).
- Python 3 + matplotlib / numpy: gráficos dos experimentos.
- Pillow (PIL): montagem dos GIFs da demonstração.
- python-pptx + reportlab: geração automática dos slides (PPTX e PDF).
- Bash: run_all.sh automatiza a suíte; getrusage mede CPU/I/O.
- Git/GitHub: versionamento. Validação cruzada: Notebook + servidor Titan (USP).

Claude (IA) — apoio, NÃO o autor

- Usado como assistente de apoio, revisão e incremento — a lógica da Árvore B (busca, split, remoção) é autoral.
- Apoiou no desenho do MENU interativo (organização das opções e mensagens).
- Apoiou na EXPORTAÇÃO DE GRAFOS (integração com Graphviz) e nos scripts de gráficos/GIF.
- Revisão de código e de texto: clareza dos comentários, consistência e checagem de erros.
- Incrementos de apresentação: estes slides, o roteiro e as tabelas foram gerados/revisados com apoio da IA.
- Princípio: a IA acelerou ferramentas acessórias; o entendimento e as decisões do trabalho são nossos.

35. Conclusão

- Implementamos uma Árvore B que opera estritamente em disco, com 1 nó por I/O — exatamente como o enunciado pede.
- Achado 1: o I/O por operação cai com m até SATURAR ($\sim m=64-128$). Existe um m ótimo — o nó dimensionado para um bloco de disco.
- Achado 2: o reaproveitamento de nós (free list) economiza $\sim 27-30\%$ do tamanho do arquivo no workload de churn.
- Achado 3: resultados DETERMINÍSTICOS — acessos a disco idênticos em 2 máquinas (Notebook x Titan/USP); só o tempo varia.
- Mensagem final: a Árvore B é rasa, balanceada e econômica em acessos — por isso é a base de bancos de dados e sistemas de arquivos.

36. Referências

- Comer, D. (1979). The Ubiquitous B-Tree. ACM Computing Surveys, 11(2), 121-137.
- Knuth, D. The Art of Computer Programming, Vol. 3 — Sorting and Searching.
- Folk, M. & Zoellick, B. File Structures.
- Bayer, R. & McCreight, E. (1972). Organization and Maintenance of Large Ordered Indexes.
- Baranauskas, J. A. Slides AED-PG-2026 (Árvores B). DCM / USP.
- Obrigado! Ficamos à disposição para perguntas.